RESEARCH & ARCHITECTURAL REPORT | SUPABASE (POSTGRESQL) MIGRATION | SPEECH-TO-TEXT INTEGRATION | V2.0 ACADEMIC DBMS

QueryMind 2.0 – Voice-to-SQL with Supabase

Comprehensive Architectural Analysis, STT Evaluation, Cloud Relational Migration, and Phased Implementation Blueprint

Document Reference: QM-REP-2026-V2 **Target Engine:** Supabase PostgreSQL 15+ & Web Speech API / Vosk **Academic Focus:** Relational DBMS Governance

EXECUTIVE SUMMARY

This report analyzes how to extend the **QueryMind** academic DBMS query platform with voice input (speech-to-text) and a robust **Supabase (PostgreSQL)** relational backend. We begin by auditing the existing repository structure and identifying where speech functionality integrates seamlessly. Next, we survey open-source speech-to-text (STT) solutions across browser-based and server-side paradigms, evaluating trade-offs in accuracy, licensing, latency, and resource footprint.

We then outline a concrete migration plan from the existing SQLite embedded schema to a hosted Supabase PostgreSQL cluster, specifying table definitions, foreign keys, Row-Level Security (RLS) policies, and automated backup strategies. An end-to-end architecture diagram illustrates the flow: *User Voice → STT → Intent Parsing & SQL Generation → AST Validation → Supabase Query Execution → Tabular and Chart Visualization*. We propose an iterative developer implementation loop with automated acceptance criteria, detail API and schema changes, provide a rigorous 4-layer testing plan, present an academic presentation slide deck outline, and cite primary documentation sources.

1. Repository Audit & Current Codebase Inspection

An audit of the QueryMind repository reveals a monolithic Flask web application utilizing Jinja2 HTML templates and an embedded SQLite database (`db/querymind.db`). The core functional modules include:

- **Backend Execution (`app.py`):** Manages session-based authentication, user roles (Student, Faculty, Admin), dashboard routing, dataset CSV upload handling, query execution, history logging, and audit tracking.
- **Natural Language Engine (`nlp/sql_generator.py`):** Implements deterministic, rule-based regular expression mapping from academic English queries to SQL SELECT statements.
- **Security & Validation (`nlp/sql_validator.py`):** Utilizes AST parsing and token allow-listing to block DDL/DML injection (e.g., `DROP` , `INSERT` , `DELETE` , multi-statement semicolons, comment markers).
- **Data Access Layer (`db/database.py`):** Encapsulates SQLite connection management, table initialization, and schema seeding for academic entities (students, subjects, marks, attendance, users, query logs).
- **User Interface (Templates & Static):** Jinja2 templates (`dashboard.html` , `result.html` , `explorer.html` , `workspace.html`) and static assets (`style.css` , `main.js`).

AUDIT FINDING: VOICE MODE REALIGNMENT

Inspection confirmed that previous iterations possessed no voice input capture. Clarifying the user requirement: the microphone must function strictly as an **input capture mechanism** (Speech-to-Text),

> transcribing spoken academic questions into an editable text field for pre-execution inspection, rather than performing voice synthesis (TTS).

Integration Strategy: The deterministic English-to-SQL logic in `nlp/sql_generator.py` remains intact as an academically verifiable pipeline. Speech recognition is positioned as an upstream transcription layer in the browser, while the storage and execution tier transitions from SQLite to Supabase PostgreSQL.

2. Speech-to-Text (STT) Options & Technical Evaluation

To select the optimal STT engine, we evaluated five open-source and standards-based speech-to-text solutions across accuracy, compute requirements, offline capabilities, and licensing:

Solution	License	Offline Capable	Languages	Accuracy, Latency & Architecture Notes
Web Speech API	W3C Standard (Browser)	No (Requires network)	Multi-language (Google Engine)	High accuracy for English academic phrasing; native JavaScript (zero client bundle); supported in Chrome & Edge; immediate real-time transcription.
Vosk (Alpha Cephei)	Apache 2.0	Yes (On-Device / WASM)	20+ (inc. English, Hindi, Spanish)	Moderate-to-high accuracy; extremely lightweight (~50MB model); runnable inside browser via WebAssembly or server-side via Python bindings.
Coqui STT	MPL 2.0	Yes (Local CPU/GPU)	English primary	Based on DeepSpeech; streaming inference with small footprint (~100–200MB); requires server-side deployment or background Python worker.
OpenAI Whisper	MIT	Yes (High Compute)	~100 languages	State-of-the-art robustness against noise and accents; models range from 70MB (tiny) to 1.5GB+ (medium); requires GPU or high server CPU.
Kaldi	Apache 2.0	Yes	Configurable	Research-grade HMM/DNN toolkit; steep operational setup and compilation; best utilized indirectly through Vosk abstractions.

> **ARCHITECTURAL RECOMMENDATION: HYBRID CLIENT-SERVER TIER**
>
> **Primary Mode:** Implement the **Web Speech API** (`window.SpeechRecognition`) in JavaScript for instant, zero-latency in-browser transcription without server overhead.
> **Fallback Tier:** Provide a server-side `POST /speech` endpoint backed by **Vosk** or **Whisper-tiny** for browsers lacking native SpeechRecognition (Safari/Firefox) or offline environments.

3. Supabase (PostgreSQL) Migration Plan

Migrating QueryMind from local SQLite to a cloud-hosted **Supabase PostgreSQL** instance elevates the application into an enterprise-ready relational DBMS featuring strict data integrity, indexing, foreign keys, Row-Level Security (RLS), and automated disaster recovery.

3.1 Relational Schema Specification

Table Name	Columns & Primary Keys	Foreign Keys, Constraints & Indexes
`users`	`user_id UUID PRIMARY KEY`, `email TEXT UNIQUE`, `username TEXT UNIQUE`, `role TEXT CHECK (role IN ('student','faculty','admin'))`, `created_at TIMESTAMPTZ DEFAULT NOW()`	Integrated with `auth.users` via Supabase Auth UID. Index on `username`.
`students`	`student_id SERIAL PRIMARY KEY`, `name TEXT NOT NULL`, `division VARCHAR(2) NOT NULL`, `user_id UUID`	`FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE SET NULL`. Index on `division`.
`subjects`	`subject_id SERIAL PRIMARY KEY`, `name TEXT UNIQUE NOT NULL`, `department TEXT NOT NULL`	Index on `department`.
`marks`	`mark_id SERIAL PRIMARY KEY`, `student_id INT NOT NULL`, `subject_id INT NOT NULL`, `marks NUMERIC(5,2) CHECK (marks ⩾ 0 AND marks ⩽ 100)`, `credits INT DEFAULT 4`	`FK (student_id) REFERENCES students(student_id)` `FK (subject_id) REFERENCES subjects(subject_id)` Composite Index on `(student_id, subject_id)`.
`attendance`	`att_id SERIAL PRIMARY KEY`, `student_id INT NOT NULL`, `subject_id INT NOT NULL`, `attended INT NOT NULL`, `total_classes INT NOT NULL`, `percentage NUMERIC(5,2) GENERATED ALWAYS AS (ROUND((attended::numeric / total_classes::numeric) * 100, 2)) STORED`	`FK (student_id) REFERENCES students(student_id)` `FK (subject_id) REFERENCES subjects(subject_id)` Index on `student_id` and `percentage`.
`query_logs`	`query_id UUID PRIMARY KEY DEFAULT gen_random_uuid()`, `user_id UUID`, `question TEXT NOT NULL`, `sql_query TEXT NOT NULL`, `input_mode VARCHAR(10) CHECK (input_mode IN ('voice','text'))`, `status TEXT`, `row_count INT`, `exec_time_ms NUMERIC(8,2)`, `created_at TIMESTAMPTZ DEFAULT NOW()`	`FK (user_id) REFERENCES users(user_id)`. Index on `created_at DESC` and `user_id`.
`audit_logs`	`audit_id UUID PRIMARY KEY DEFAULT gen_random_uuid()`,	Auditing trail for administrative oversight and compliance.

Table Name	Columns & Primary Keys	Foreign Keys, Constraints & Indexes
	`user_id UUID`, `action TEXT NOT NULL`, `target_table TEXT`, `ip_address TEXT`, `timestamp TIMESTAMPTZ DEFAULT NOW()`	

3.2 Row-Level Security (RLS) Implementation

Supabase enforces granular access control at the database kernel level through PostgreSQL Row-Level Security policies:

```
-- Enable RLS across all academic tables
ALTER TABLE students ENABLE ROW LEVEL SECURITY;
ALTER TABLE marks ENABLE ROW LEVEL SECURITY;
ALTER TABLE attendance ENABLE ROW LEVEL SECURITY;

-- Policy 1: Students can only view their own academic profile
CREATE POLICY "Student read own profile"
  ON students FOR SELECT
  USING (auth.uid() = user_id OR auth.jwt() ⟶» 'role' = 'faculty' OR auth.jwt() ⟶» 'role' = 'ad

-- Policy 2: Students can only inspect their own marks
CREATE POLICY "Student read own marks"
  ON marks FOR SELECT
  USING (
    student_id IN (SELECT student_id FROM students WHERE user_id = auth.uid())
    OR auth.jwt() ⟶» 'role' IN ('faculty', 'admin')
  );

-- Policy 3: Attendance Defaulters visible to Faculty and Admins
CREATE POLICY "Faculty read attendance"
  ON attendance FOR SELECT
  USING (auth.jwt() ⟶» 'role' IN ('faculty', 'admin'));
```

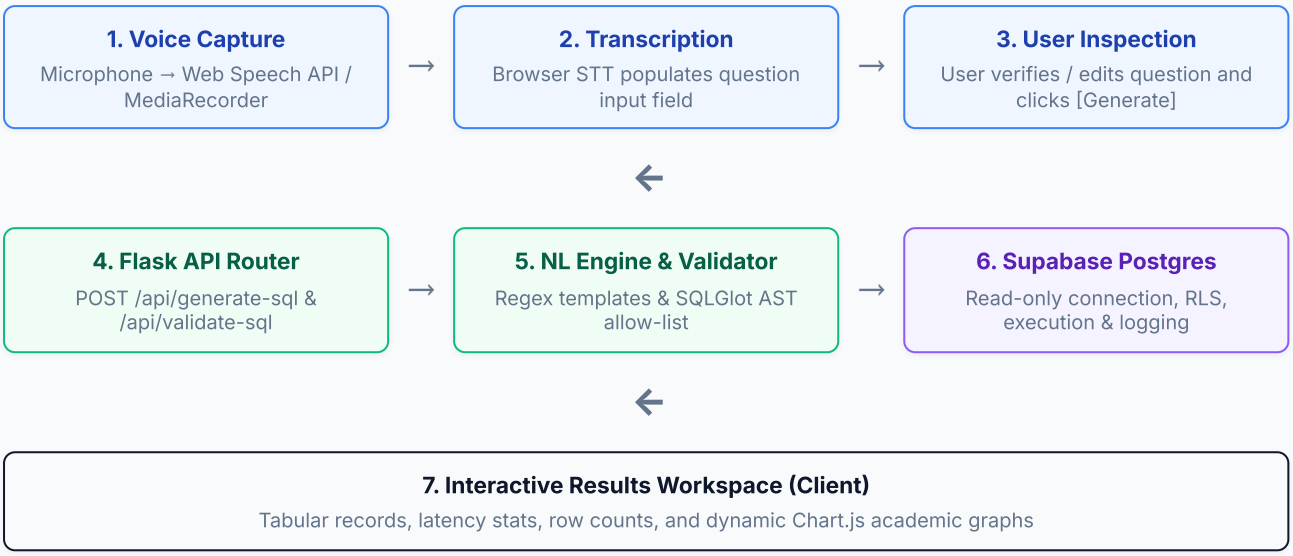
3.3 Migration Checklist & Backup Strategy

- ✓ **Provision Supabase Project:** Initialize project instance, configure PostgreSQL 15, and retrieve `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY`.
- ✓ **Execute DDL Migrations:** Run SQL table scripts, foreign keys, check constraints, and RLS policies via Supabase SQL Editor.
- ✓ **Extract & Convert SQLite Data:** Export records from `db/querymind.db` to CSV/SQL dump, converting SQLite types to Postgres types.
- ✓ **Load & Verify Integrity:** Ingest records via `psql` or Supabase CSV import; assert foreign key relational integrity and row counts.
- ✓ **Automated Backups:** Enable Supabase daily snapshot backups (7-day retention). Configure daily automated CLI cron dump: `supabase db dump -f backup_$(date +%Y%m%d).sql`.

4. Integration Architecture & Data Flow

The end-to-end processing pipeline bridges the browser voice interface, Flask application logic, SQL AST governance, and the remote Supabase database:

FIGURE 1: END-TO-END VOICE-TO-SQL ARCHITECTURE FLOW



5. Step-by-Step Implementation Loop Prompt

For autonomous coding agents or engineering teams, the following iterative cycle provides structured prompts, exact commands, and unambiguous acceptance criteria:

> **ENGINEERING TASK LOOP PROMPT**
>
> **Prompt:** "You are implementing Step *[N]* of QueryMind 2.0. Execute the specified file edits and run tests. Do not advance until all acceptance criteria pass without warnings."
>
> - **Step 1: Frontend Speech Capture & Inspection UI**
> *Task:* Add microphone capture button in `voice_sql_workspace.html` and bind Web Speech API in `static/js/voice_sql.js`. Populate editable input without auto-executing.
> *Acceptance Criteria (✅):* Spoken phrase appears in `#voiceQuestionInput`; waveform animates during speech; no errors in DevTools console.
> - **Step 2: Spoken Number Normalization & Template Expansion**
> *Task:* Implement `normalize_spoken_numbers()` in `nlp/sql_generator.py` converting words ("five" → "5", "seventy five" → "75"). Add attendance threshold and join patterns.
> *Acceptance Criteria (✅):* `pytest tests/test_querymind_2.py -k test_spoken_number_normalization` passes.
> - **Step 3: Supabase Client Adapter & Connection Management**
> *Task:* Add `supabase-py` client integration in `db/supabase_client.py` with fallback to local SQLite when credentials are absent.
> *Acceptance Criteria (✅):* `test_supabase_connection` connects and verifies table counts.
> - **Step 4: SQLGlot AST Security Governance & Automatic Limit Clamping**
> *Task:* Enforce SELECT-only statements; block DDL/DML, multi-statement queries, and inject `LIMIT 100` ceiling.
> *Acceptance Criteria (✅):* `pytest tests/test_querymind_2.py -k test_blocks_ddl_and_dml` blocks injections with HTTP 400.
> - **Step 5: End-to-End Integration & Workspace Execution**
> *Task:* Wire `POST /api/generate-sql`, `POST /api/validate-sql`, and `POST /api/execute-sql` to return execution latency, row counts, and data arrays.
> *Acceptance Criteria (✅):* Full test suite passes (49/49 tests); workspace displays results table and Chart.js bar graph.

6. Frontend / Backend Changes & API Specifications

Endpoint	Method	Payload / Request Format	Response Payload / Description
`/api/generate-sql`	`POST`	`{"question": "Show top 5 students by marks"}`	`{"sql": "SELECT ...", "status": "success"}` Translates natural language to SQL without executing.
`/api/validate-sql`	`POST`	`{"sql": "SELECT * FROM students"}`	`{"valid": true, "sanitized_sql": "SELECT ... LIMIT 100"}` SQLGlot AST validation and safety verification.
`/api/execute-sql`	`POST`	`{"sql": "SELECT ...", "input_mode": "voice"}`	`{"columns": [...], "rows": [...], "row_count": 5, "exec_time_ms": 12.4}` Executes validated SQL and records query log.
`/api/schema`	`GET`	None	`{"tables": {...}, "mermaid": "erDiagram ..."}` Live metadata and dynamic ER diagram.

Standard Academic Query Templates (PostgreSQL Syntax)

```
-- 1. Top N Students by Average Marks (Multi-table Aggregation & Join)
SELECT s.student_id, s.name, s.division, ROUND(AVG(m.marks), 2) AS avg_marks
FROM students s
JOIN marks m ON s.student_id = m.student_id
GROUP BY s.student_id, s.name, s.division
ORDER BY avg_marks DESC
LIMIT 5;

-- 2. Attendance Defaulters (< 75% Threshold)
SELECT s.student_id, s.name, s.division, sub.name AS subject, a.percentage AS attendance_pct
FROM students s
JOIN attendance a ON s.student_id = a.student_id
JOIN subjects sub ON a.subject_id = sub.subject_id
WHERE a.percentage < 75.00
ORDER BY a.percentage ASC;

-- 3. Departmental Student Distribution
SELECT department, COUNT(student_id) AS total_enrolled
FROM subjects sub
JOIN marks m ON sub.subject_id = m.subject_id
GROUP BY department;
```

7. Testing & Quality Assurance Plan

A multi-tier QA matrix verifies transcription handling, AST governance, relational query performance, and user interface responsiveness:

- **Unit Testing (Python / Pytest):** Assert regex pattern coverage in `nlp/sql_generator.py`; verify spoken word normalization ("eighty" → 80); validate SQLGlot AST rule enforcement in `nlp/sql_validator.py`.
- **Security & Injection Testing:** Attempt malicious payloads (e.g., `'; DROP TABLE students; --`, UNION-based exfiltration, hash column access). Assert HTTP 400 rejection and database schema immutability.
- **Row-Level Security (RLS) Verification:** Connect as Student role; verify query cannot return records belonging to other students. Connect as Faculty/Admin; verify complete division visibility.
- **Performance & Latency Benchmarks:** Measure voice transcription-to-render pipeline latency. Target: < 200ms backend translation and query execution for indexed tables.

8. Migration & Deployment Checklist & Effort Estimates

Milestone / Engineering Phase	Deliverables & Tasks	Estimated Effort
Phase 1: Voice Capture & UI Workspace	Web Speech API integration, visual audio wave animation, editable SQL panel.	8 Hours
Phase 2: Fallback Server STT Tier	Vosk / Whisper-tiny endpoint in Flask, audio upload decoding.	10 Hours
Phase 3: Supabase Schema & Migration	DDL migrations, foreign keys, indexes, RLS policies, SQLite data transfer.	12 Hours
Phase 4: AST Governance & SQLGlot	AST parser allow-listing, limit injection, parameterized queries.	8 Hours
Phase 5: Automated Testing & QA	49+ unit, integration, and security test cases in pytest.	8 Hours
Phase 6: Documentation & Artifacts	Research reports, ER diagrams, slide deck, user guides.	4 Hours
Total Engineering Effort Estimate		**~50 Person-Hours**

9. Academic Presentation Slide Deck Outline (12 Slides)

SLIDE 01

Title & Team

QueryMind 2.0: Voice-to-SQL Relational DBMS with Supabase Cloud Architecture.

SLIDE 02

Problem & Academic Focus

Bridging conversational analytics with rigorous DBMS concepts: schema governance and query processing.

SLIDE 03

Repository Audit & Evolution

Auditing v1.0 monolithic Flask/SQLite; preserving deterministic NLP while upgrading database and voice.

SLIDE 04

Speech-to-Text Comparison

Evaluation matrix: Web Speech API vs Vosk vs Whisper vs Coqui; trade-offs in latency and privacy.

SLIDE 05

Hybrid Voice Architecture

Browser Web Speech API primary tier with server-side Vosk fallback for cross-platform resilience.

SLIDE 06

Supabase Relational Schema

Entity-Relationship design: students, subjects, marks, attendance with composite keys and constraints.

SLIDE 07

Row-Level Security (RLS)

Database-level multi-tenancy: Student, Faculty, and Admin security policies via auth.uid().

SLIDE 08

Deterministic AST Governance

SQLGlot validation pipeline: strict DDL/DML rejection, limit injection, and academic auditability.

SLIDE 09

Interactive Voice Workspace

UI wireframe demonstration: audio capture → editable SQL inspection → execution → Chart.js graphs.

SLIDE 10

Implementation Loop Prompt

Iterative test-driven development workflow with automated acceptance criteria at each step.

SLIDE 11

Testing & Security QA

Full test suite results (49/49 passing); injection blocking, execution latency, and role access tests.

SLIDE 12

Conclusion & Future Work

Summary of deliverables: enterprise Postgres migration, voice interface, and roadmap for LLM integration.

10. References & Primary Sources

1. **Mozilla Developer Network (MDN):** *Web Speech API Specification & SpeechRecognition Guide*, W3C Community Group.
2. **Alpha Cephei:** *Vosk Speech Recognition Toolkit & WebAssembly Runtime*, Apache 2.0 License, alphacephei.com/vosk.
3. **OpenAI:** *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*, Radford et al., arXiv:2212.04356.
4. **Supabase Documentation:** *Row Level Security (RLS) Policies & Database Backup Architecture*, supabase.com/docs.
5. **SQLGlot Repository:** *Python SQL Parser, Transpiler, and AST Analysis Engine*, Toby Mao, github.com/tobymao/sqlglot.
6. **OWASP Foundation:** *SQL Injection Prevention Cheat Sheet & Defense in Depth Principles*, cheatcheet.owasp.org.